

Program No. 1

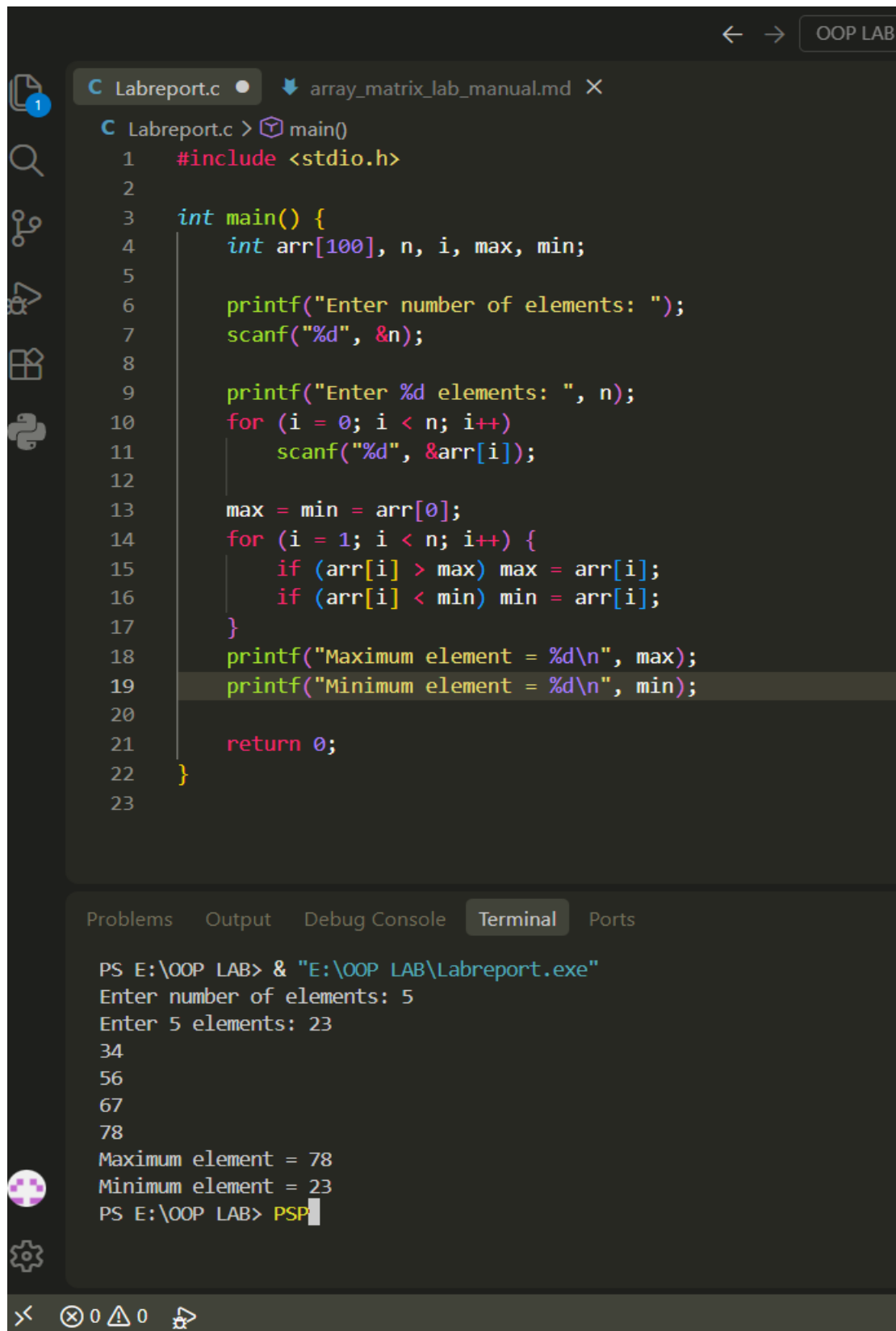
Program Name: Find Maximum / Minimum in an Array.

Objective: To write a C program to find the maximum and minimum element present in a given array.

Procedure:

1. Declare and initialize an array of n elements.
2. Assume the first element as both the maximum and minimum.
3. Traverse the array from the second element to the last.
4. Compare each element with the current maximum; update if a larger value is found.
5. Compare each element with the current minimum; update if a smaller value is found.
6. Print the maximum and minimum values after the traversal.

Input&Output:



The image shows a code editor window with a file named `Labreport.c` open. The code is a C program that finds the maximum and minimum elements in an array. The code is as follows:

```
1  #include <stdio.h>
2
3  int main() {
4      int arr[100], n, i, max, min;
5
6      printf("Enter number of elements: ");
7      scanf("%d", &n);
8
9      printf("Enter %d elements: ", n);
10     for (i = 0; i < n; i++)
11         scanf("%d", &arr[i]);
12
13     max = min = arr[0];
14     for (i = 1; i < n; i++) {
15         if (arr[i] > max) max = arr[i];
16         if (arr[i] < min) min = arr[i];
17     }
18     printf("Maximum element = %d\n", max);
19     printf("Minimum element = %d\n", min);
20
21     return 0;
22 }
```

The terminal output shows the execution of the program:

```
PS E:\OOP LAB> & "E:\OOP LAB\Labreport.exe"
Enter number of elements: 5
Enter 5 elements: 23
34
56
67
78
Maximum element = 78
Minimum element = 23
PS E:\OOP LAB> PSP
```

Fig-Input&Output(Find Maximum / Minimum in an Array)

Observation: For input `arr = {23,34,56,67,78}` the program correctly identifies 78 as the maximum and 23 as the minimum by comparing each element sequentially against the running max/min values.

Conclusion: The maximum and minimum elements of an array can be found in a single linear pass ($O(n)$ time complexity) using two variables that track the running maximum and minimum.

Program No. 2

Program Name: Insert an Element in an Array.

Objective: To write a C program to insert a new element at a specified position in an array.

Procedure:

1. Read the array elements and the current size n.
2. Read the value to be inserted and the position pos at which it should be inserted.
3. Starting from the last element, shift each element one position to the right up to pos.
4. Place the new value at position pos.
5. Increase the size of the array by 1 and display the updated array.

Input&Output:

```
C Labreport.c •
C Labreport.c > main()
1  #include <stdio.h>
2
3  int main() {
4      int arr[100], n, i, pos, value;
5
6      printf("Enter number of elements: ");
7      scanf("%d", &n);
8
9      printf("Enter %d elements: ", n);
10     for (i = 0; i < n; i++)
11         scanf("%d", &arr[i]);
12
13     printf("Enter position to insert (1 to %d): ", n + 1);
14     scanf("%d", &pos);
15
16     printf("Enter value to insert: ");
17     scanf("%d", &value);
18
19     for (i = n; i >= pos; i--)
20         arr[i] = arr[i - 1];
21
22     arr[pos - 1] = value;
23     n++;
24
25     printf("Array after insertion: ");
26     for (i = 0; i < n; i++)
27         printf("%d ", arr[i]);
28     printf("\n");
29
30     return 0;
31 }
32
```

Problems Output Debug Console Terminal Ports

```
PS E:\OOP LAB> & "E:\OOP LAB\Labreport.exe"
Enter number of elements: 5
Enter 5 elements: 10
20
30
40
50
Enter position to insert (1 to 6): 4
Enter value to insert: 33
Array after insertion: 10 20 30 33 40 50
PS E:\OOP LAB> 
```

Fig-Input&Output(Insert an element in an array)

Observation: For $\text{arr} = \{10, 20, 30, 40, 50\}$ and inserting 33 at position 4, the elements 40 and 50 shift right, giving the result $\{10, 20, , 30, 33, 40, 50\}$.

Conclusion: Inserting an element into an array requires shifting all subsequent elements to the right, which takes $O(n)$ time in the worst case since arrays occupy contiguous memory.

Program No. 3

Program Name: Delete an Element in an Array.

Objective: To write a C program to delete an element at a specified position from an array.

Procedure:

1. Read the array elements and size n.
2. Read the position pos from which the element should be deleted.
3. Shift all elements after pos one place to the left.
4. Decrease the size of the array by 1.
5. Display the resulting array.

Input&output:

```
C Labreport.c X
C Labreport.c > main()
1  #include <stdio.h>
2
3  int main() {
4      int arr[100], n, i, pos;
5
6      printf("Enter number of elements: ");
7      scanf("%d", &n);
8
9      printf("Enter %d elements: ", n);
10     for (i = 0; i < n; i++)
11         scanf("%d", &arr[i]);
12
13     printf("Enter position to delete: ", n);
14     scanf("%d", &pos);
15
16     for (i = pos - 1; i < n - 1; i++)
17         arr[i] = arr[i + 1];
18
19     n--;
20
21     printf("Array after deletion: ");
22     for (i = 0; i < n; i++)
23         printf("%d ", arr[i]);
24     printf("\n");
25
26     return 0;
27 }
```

Problems Output Debug Console **Terminal** Ports

```
PS E:\OOP LAB> & "E:\OOP LAB\Labreport.exe"
Enter number of elements: 5
Enter 5 elements: 10
20
30
40
50
Enter position to delete: 3
Array after deletion: 10 20 40 50
PS E:\OOP LAB> |
```

Fig-Input&Output(Delete an Element in an array).

Observation: For $\text{arr} = \{10, 20, 30, 40, 50\}$ and deleting the element at position 3 (value 30), the elements after it shift left, giving $\{10, 20, 40, 50\}$.

Conclusion: Deleting an element from an array requires left-shifting all subsequent elements by one position, which also takes $O(n)$ time in the worst case.

Program No. 4

Program Name: Delete First Occurrence in an Array.

Objective: To write a C program to delete the first occurrence of a given value from an array.

Procedure:

1. Read the array and its size n.
2. Read the value to be deleted.
3. Traverse the array from the beginning to find the index of the first occurrence of that value.
4. If found, shift all elements after that index one place to the left and reduce n by 1.
5. If not found, display an appropriate message.
6. End the program.

Input&Output:

```
C Labreport.c > main()
5
6     printf("Enter number of elements: ");
7     scanf("%d", &n);
8
9     printf("Enter %d elements: ", n);
10    for (i = 0; i < n; i++)
11        scanf("%d", &arr[i]);
12
13    printf("Enter value to delete (first occurrence): ");
14    scanf("%d", &value);
15
16    for (i = 0; i < n; i++) {
17        if (arr[i] == value) {
18            pos = i;
19            break;
20        }
21    }
22
23    if (pos == -1) {
24        printf("Value not found in array.\n");
25    } else {
26        for (i = pos; i < n - 1; i++)
27            arr[i] = arr[i + 1];
28        n--;
29
30        printf("Array after deleting first occurrence: ");
31        for (i = 0; i < n; i++)
32            printf("%d ", arr[i]);
33        printf("\n");
34    }
35
36    return 0;
37 }
```

Problems Output Debug Console Terminal Ports

```
PS E:\OOP LAB> cd "e:\OOP LAB\" ; if ($?) { gcc Labreport.c -o Labreport } ;
Enter number of elements: 5
Enter 5 elements: 10
20
30
40
20
Enter value to delete (first occurrence): 20
Array after deleting first occurrence: 10 30 40 20
PS E:\OOP LAB> 
```

Fig-Input&Output(Delete First Occurrence in an Array)

Observation:

- Input: 10 20 30 40 20 delete first occurrence of 20
- Output: 10 30 40 20
- Only the first matching element (index 1) was removed; the later 20 remains.

Conclusion: Deleting the first occurrence requires a forward linear search to locate the earliest matching index before shifting elements.

Program No: 5

Program Name: Delete Last Occurrence in an Array.

Objective: To write a C program to delete the last occurrence of a given value in an array.

Procedure:

1. Start.
2. Read array size n, elements, and value to delete.
3. Search the array from the end for the last index where value matches.
4. If found, shift all elements after that index one place left.
5. Decrease the array size by 1.
6. If not found, print a message.
7. Stop.

Input&Output:

```
C Labreport.c
C Labreport.c > main()
1  #include <stdio.h>
2
3  int main() {
4      int arr[100], n, i, value, pos = -1;
5
6      printf("Enter number of elements: ");
7      scanf("%d", &n);
8
9      printf("Enter %d elements: ", n);
10     for (i = 0; i < n; i++)
11         scanf("%d", &arr[i]);
12
13     printf("Enter value to delete: ");
14     scanf("%d", &value);
15
16     for (i = n - 1; i >= 0; i--) {
17         if (arr[i] == value) {
18             pos = i;
19             break;
20         }
21     }
22
23     if (pos == -1) {
24         printf("Value not found in array.\n");
25     } else {
26         for (i = pos; i < n - 1; i++)
27             arr[i] = arr[i + 1];
28         n--;
29
30         printf("Array after deleting last occurrence: ");
31         for (i = 0; i < n; i++)
32             printf("%d ", arr[i]);
33         printf("\n");
34     }
35     return 0;
36 }
```

Problems Output Debug Console Terminal Ports

```
PS E:\OOP LAB> & "E:\OOP LAB\Labreport.exe"
Enter number of elements: 5
Enter 5 elements: 10
20
30
40
20
Enter value to delete: 20
Array after deleting last occurrence: 10 20 30 40
PS E:\OOP LAB>
```

Fig-Input&Output(Last Occurrence in an Array)

Observation:

- Input: 10 20 30 40 20, delete last occurrence of 20
- Output: 10 20 30 40
- The 20 at index 3 was removed while the earlier 20 at index 1 remains.

Conclusion: Deleting the last occurrence requires a backward linear search from the end of the array to correctly identify the last matching index.

Program No: 6

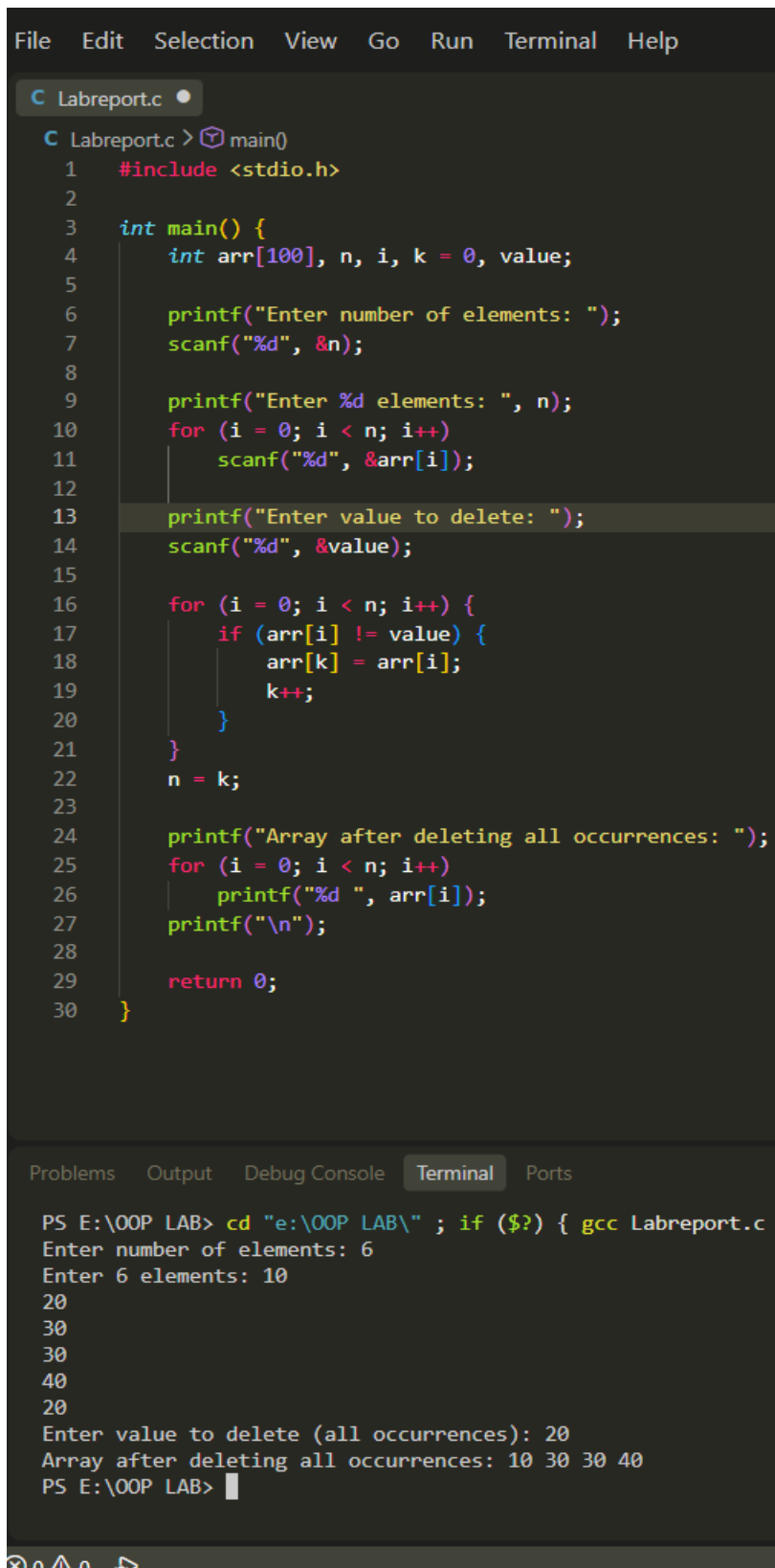
Program Name: Delete All Occurrences in an Array

Objective: To write a C program to delete all occurrences of a given value from an array.

Procedure:

1. Start.
2. Read array size n, elements, and value to delete.
3. Create a new index counter k = 0.
4. Traverse the array; for every element not equal to the value, copy it to arr[k] and increment k.
5. Set the new size of array as k.
6. Print the updated array.
7. Stop.

Input&Output:



The image shows a code editor with a C program named 'Labreport.c' and a terminal window showing its execution. The program is designed to delete all occurrences of a specified value from an array.

```
C Labreport.c > main()
1  #include <stdio.h>
2
3  int main() {
4      int arr[100], n, i, k = 0, value;
5
6      printf("Enter number of elements: ");
7      scanf("%d", &n);
8
9      printf("Enter %d elements: ", n);
10     for (i = 0; i < n; i++)
11         scanf("%d", &arr[i]);
12
13     printf("Enter value to delete: ");
14     scanf("%d", &value);
15
16     for (i = 0; i < n; i++) {
17         if (arr[i] != value) {
18             arr[k] = arr[i];
19             k++;
20         }
21     }
22     n = k;
23
24     printf("Array after deleting all occurrences: ");
25     for (i = 0; i < n; i++)
26         printf("%d ", arr[i]);
27     printf("\n");
28
29     return 0;
30 }
```

The terminal output shows the program being compiled and executed. It prompts the user to enter the number of elements (6) and the elements themselves (10, 20, 30, 30, 40, 20). It then prompts for the value to delete (20) and displays the resulting array (10 30 30 40).

```
PS E:\OOP LAB> cd "e:\OOP LAB\" ; if ($?) { gcc Labreport.c
Enter number of elements: 6
Enter 6 elements: 10
20
30
30
40
20
Enter value to delete (all occurrences): 20
Array after deleting all occurrences: 10 30 30 40
PS E:\OOP LAB>
```

Fig-Input&Output(Delete All Occurrences in an Array)

Observation:

- Input: 10 20 30 30 40 20, delete all occurrences of 20
- Output: 10 30 30 40
- All three occurrences of 20 were removed in a single pass.

Conclusion: Using a separate write-index (k) allows all occurrences of a value to be removed in one linear pass, giving $O(n)$ time complexity.

Program No: 7

Program Name: Move All 0s to the Front

Objective: To write a C program that rearranges an array so that all zero elements appear at the front, keeping the relative order of non-zero elements.

Procedure:

1. Start.
2. Read array size n and elements.
3. Take an index $j = n - 1$ (fill from the back for non-zero elements).
4. Traverse the array from the last index to the first.
5. If the current element is non-zero, place it at index j in a temporary array and decrement j.
6. Fill remaining indices from 0 to j with 0.
7. Print the updated array.
8. Stop.

Input&Output:

```
File Edit Selection View Go Run Terminal Help

C Labreport.c X
C Labreport.c > main()
1  #include <stdio.h>
2
3  int main() {
4      int arr[100], temp[100], n, i, j;
5
6      printf("Enter number of elements: ");
7      scanf("%d", &n);
8
9      printf("Enter %d elements: ", n);
10     for (i = 0; i < n; i++)
11         scanf("%d", &arr[i]);
12
13     j = n - 1;
14     for (i = n - 1; i >= 0; i--) {
15         if (arr[i] != 0) {
16             temp[j] = arr[i];
17             j--;
18         }
19     }
20     for (; j >= 0; j--)
21         temp[j] = 0;
22
23     printf("Array after moving 0s to front: ");
24     for (i = 0; i < n; i++)
25         printf("%d ", temp[i]);
26     printf("\n");
27
28     return 0;
29 }

Problems Output Debug Console Terminal Ports
PS E:\OOP LAB> & "E:\OOP LAB\Labreport.exe"
Enter number of elements: 5
Enter 5 elements: 10
20
30
0
0
0
Array after moving 0s to front: 0 0 10 20 30
PS E:\OOP LAB>
```

Fig-Input&Output(Move All 0s to the Front)

Observation:

- **Input:** 10 20 30 0 0
- **Output:** 0 0 10 20 30
- Relative order of non-zero elements (10,20,30) is preserved.

Conclusion: By filling non-zero elements from the back of a temporary array and padding the front with zeros, all zeros can be moved to the front while preserving the relative order of other elements.

Program No: 8

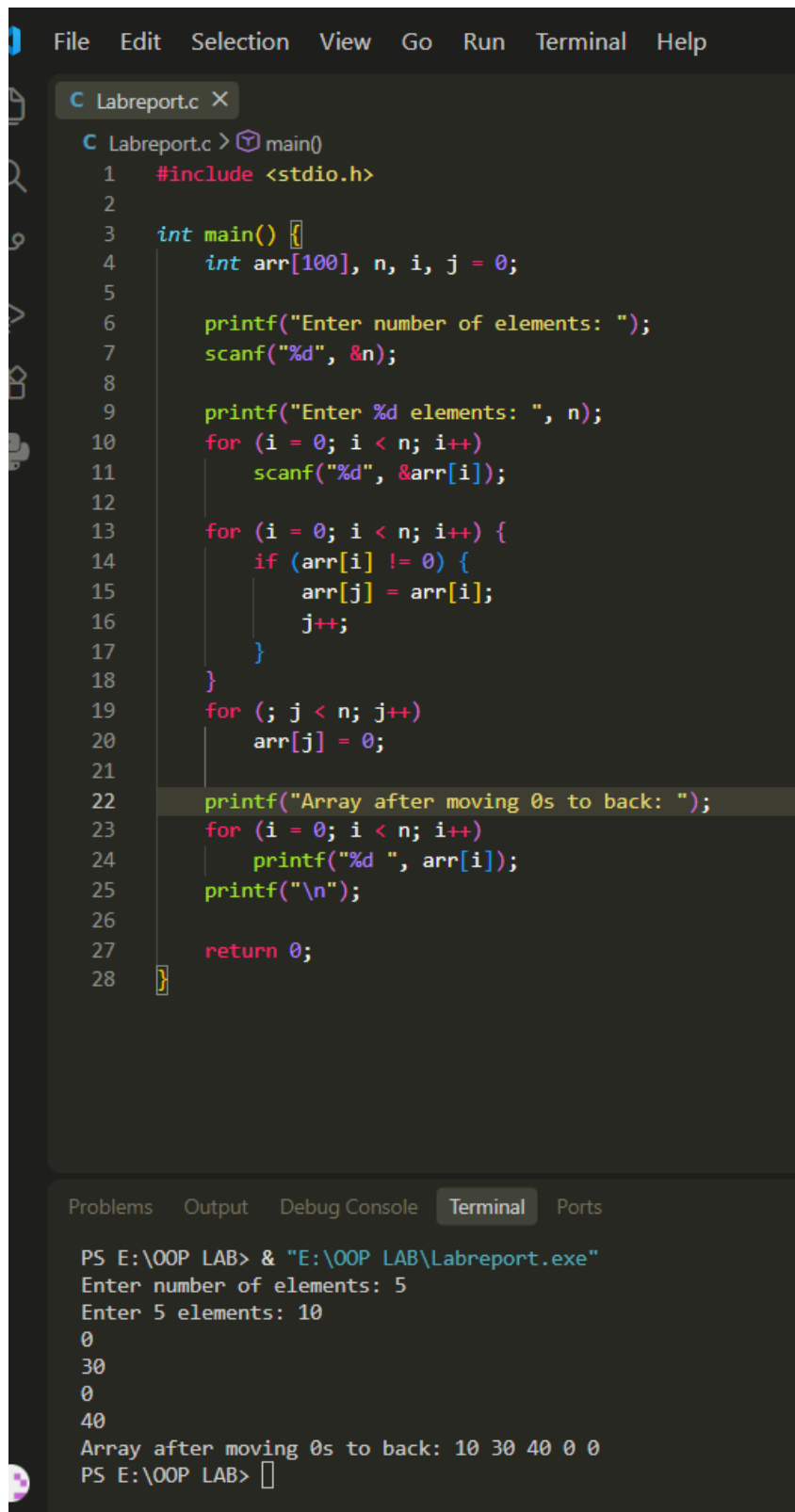
Program Name: Move All 0s to the Back

Objective: To write a C program that rearranges an array so that all zero elements appear at the end, keeping the relative order of non-zero elements.

Procedure:

1. Start.
2. Read array size n and elements.
3. Take an index $j = 0$ (fill from the front for non-zero elements).
4. Traverse the array from the first index to the last.
5. If the current element is non-zero, place it at index j and increment j.
6. Fill remaining indices from j to n-1 with 0.
7. Print the updated array.
8. Stop.

Input&Output:



```
File Edit Selection View Go Run Terminal Help
C Labreport.c X
C Labreport.c > main()
1  #include <stdio.h>
2
3  int main() {
4      int arr[100], n, i, j = 0;
5
6      printf("Enter number of elements: ");
7      scanf("%d", &n);
8
9      printf("Enter %d elements: ", n);
10     for (i = 0; i < n; i++)
11         scanf("%d", &arr[i]);
12
13     for (i = 0; i < n; i++) {
14         if (arr[i] != 0) {
15             arr[j] = arr[i];
16             j++;
17         }
18     }
19     for (; j < n; j++)
20         arr[j] = 0;
21
22     printf("Array after moving 0s to back: ");
23     for (i = 0; i < n; i++)
24         printf("%d ", arr[i]);
25     printf("\n");
26
27     return 0;
28 }
```

Problems Output Debug Console Terminal Ports

```
PS E:\OOP LAB> & "E:\OOP LAB\Labreport.exe"
Enter number of elements: 5
Enter 5 elements: 10
0
30
0
40
Array after moving 0s to back: 10 30 40 0 0
PS E:\OOP LAB> 
```

Fig-Input&Output(Move All 0s to the Back)

Observation:

- Input: 10 0 30 0 40
- Output: 10 30 40 0 0
- Relative order of non-zero elements (10,30,40) is preserved.

Conclusion: By copying non-zero elements in place using a write-pointer and padding the remaining positions with zeros, all zeros can be moved to the end in $O(n)$ time.

Program No: 9**Program Name:** Rotate Array Right**Objective:** To write a C program to rotate the elements of an array to the right by d positions.

Procedure:

1. Start.
2. Read array size n, elements, and number of rotations d.
3. Compute $d = d \% n$ to handle rotations greater than n.
4. Store the last d elements in a temporary array.
5. Shift the remaining n-d elements to the right by d positions.
6. Copy the temporary array's elements to the front of the array.
7. Print the rotated array.
8. Stop.

Input&Output:

The image shows a Visual Studio Code editor with a C program named 'Labreport.c'. The program implements a right rotation of an array. It prompts the user to enter the number of elements (n) and the number of positions to rotate right (d). It then reads n integers into an array 'arr'. A temporary array 'temp' is used to store the last 'd' elements of 'arr'. These elements are then shifted to the beginning of 'arr', and the remaining elements are shifted to the right by 'd' positions. Finally, the rotated array is printed.

```
C Labreport.c
C Labreport.c > main()
1  #include <stdio.h>
2
3  int main() {
4      int arr[100], temp[100], n, d, i;
5
6      printf("Enter number of elements: ");
7      scanf("%d", &n);
8
9      printf("Enter %d elements: ", n);
10     for (i = 0; i < n; i++)
11         scanf("%d", &arr[i]);
12
13     printf("Enter number of positions to rotate right: ");
14     scanf("%d", &d);
15     d = d % n;
16
17     for (i = 0; i < d; i++)
18         temp[i] = arr[n - d + i];
19
20     for (i = n - 1; i >= d; i--)
21         arr[i] = arr[i - d];
22
23     for (i = 0; i < d; i++)
24         arr[i] = temp[i];
25
26     printf("Array after right rotation: ");
27     for (i = 0; i < n; i++)
28         printf("%d ", arr[i]);
29     printf("\n");
30
31     return 0;
32 }
```

Terminal Output:

```
PS E:\OOP LAB> & "E:\OOP LAB\Labreport.exe"
Enter number of elements: 5
Enter 5 elements: 10
20
30
40
50
Enter number of positions to rotate right: 3
Array after right rotation: 30 40 50 10 20
PS E:\OOP LAB>
```

Fig-Input&Output(Rotate Array Right)

Observation:

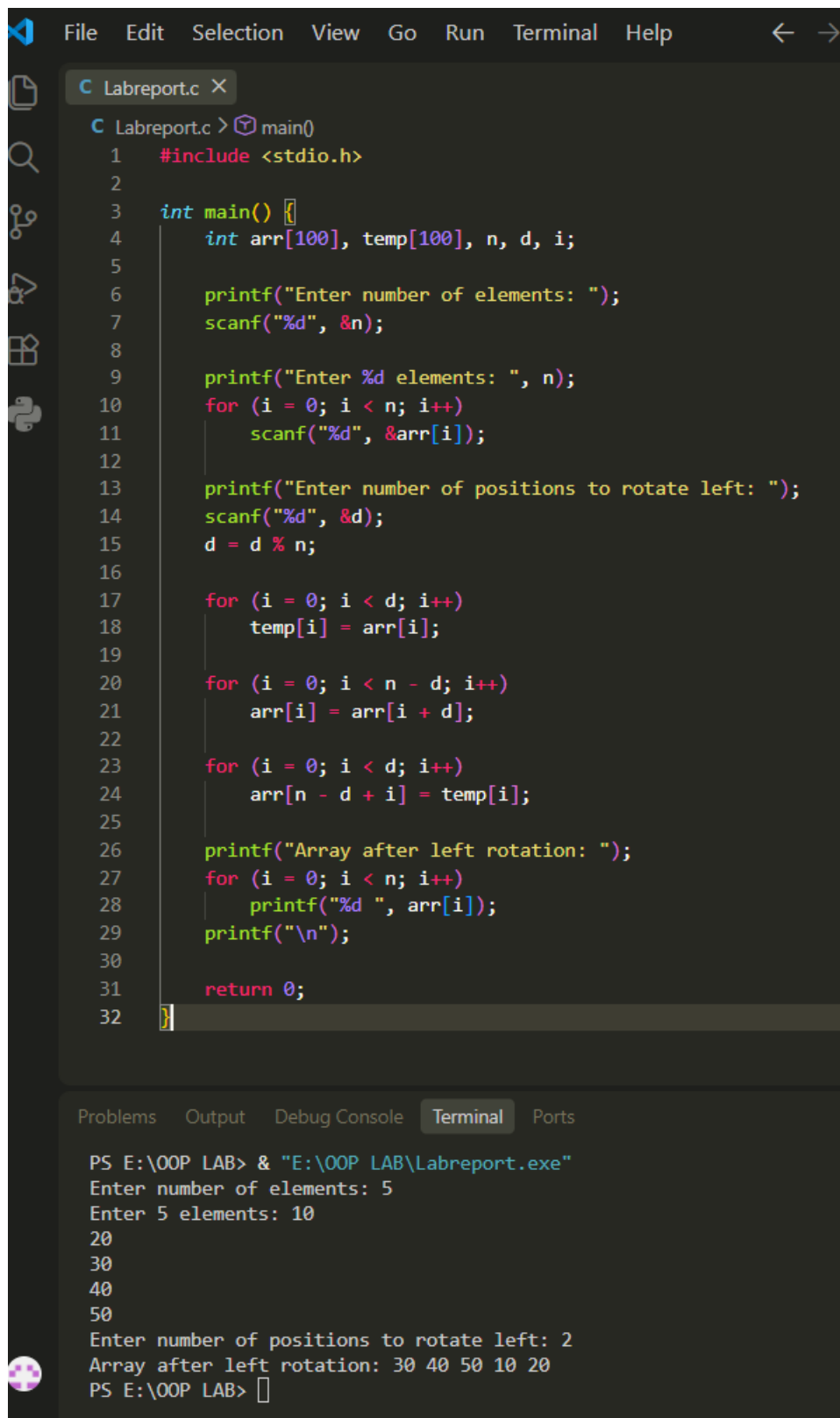
- **Input:** 10 20 30 40 50, rotate right by 3
- **Output:** 30 40 50 10 20
- The last 3 elements moved to the front, and the rest shifted right.

Conclusion: Right rotation of an array by d positions can be achieved by temporarily storing the last d elements, shifting the rest, and placing the stored elements at the front.

Program No: 10**Program Name:** Rotate Array Left**Objective:** To write a C program to rotate the elements of an array to the left by d positions.**Procedure:**

1. Start.
2. Read array size n, elements, and number of rotations d.
3. Compute $d = d \% n$ to handle rotations greater than n.
4. Store the first d elements in a temporary array.
5. Shift the remaining n-d elements to the left by d positions.
6. Copy the temporary array's elements to the end of the array.
7. Print the rotated array.
8. Stop.

Input&Output:



The image shows a screenshot of a code editor with a dark theme. The editor has a menu bar at the top with 'File', 'Edit', 'Selection', 'View', 'Go', 'Run', 'Terminal', and 'Help'. On the left side, there is a sidebar with icons for Explorer, Search, Source Control, Run and Debug, Extensions, and Testing. The main editor area displays a C program named 'Labreport.c'. The code implements a function to rotate an array to the left by a given number of positions. It uses a temporary array to store the first 'd' elements, shifts the remaining elements to the left, and then appends the stored elements to the end of the array. The program prompts the user for the number of elements, the elements themselves, and the number of positions to rotate. The output shows the array after rotation.

```
C Labreport.c X
C Labreport.c > main()
1  #include <stdio.h>
2
3  int main() {
4      int arr[100], temp[100], n, d, i;
5
6      printf("Enter number of elements: ");
7      scanf("%d", &n);
8
9      printf("Enter %d elements: ", n);
10     for (i = 0; i < n; i++)
11         scanf("%d", &arr[i]);
12
13     printf("Enter number of positions to rotate left: ");
14     scanf("%d", &d);
15     d = d % n;
16
17     for (i = 0; i < d; i++)
18         temp[i] = arr[i];
19
20     for (i = 0; i < n - d; i++)
21         arr[i] = arr[i + d];
22
23     for (i = 0; i < d; i++)
24         arr[n - d + i] = temp[i];
25
26     printf("Array after left rotation: ");
27     for (i = 0; i < n; i++)
28         printf("%d ", arr[i]);
29     printf("\n");
30
31     return 0;
32 }
```

Problems Output Debug Console Terminal Ports

```
PS E:\OOP LAB> & "E:\OOP LAB\Labreport.exe"
Enter number of elements: 5
Enter 5 elements: 10
20
30
40
50
Enter number of positions to rotate left: 2
Array after left rotation: 30 40 50 10 20
PS E:\OOP LAB>
```

Fig-Input&output(Rotate Array Left)

Observation:

- Input: 10 20 30 40 50, rotate left by 2
- Output: 30 40 50 10 20
- The first 2 elements moved to the end, and the rest shifted left.

Conclusion: Left rotation of an array by d positions can be achieved by temporarily storing the first d elements, shifting the rest left, and appending the stored elements at the end.

Program No: 11**Program Name:** Matrix Multiplication

Objective: To write a C program to multiply two matrices, provided the number of columns of the first matrix equals the number of rows of the second matrix.

Procedure:

1. Start.
2. Read dimensions of matrix A ($r_1 \times c_1$) and matrix B ($r_2 \times c_2$).
3. Read elements of matrix A and matrix B.
4. For each cell (i, j) of the resultant matrix, compute the sum of products of corresponding elements from row i of A and column j of B.
5. Store the sums in the resultant matrix C.
6. Print matrix C.
7. Stop.

Input&Output:

```
File Edit Selection View Go Run Terminal Help
C Labreport.c X
C Labreport.c > main()
1  #include <stdio.h>
2
3  int main() {
4      int a[10][10], b[10][10], c[10][10];
5      int r1, c1, r2, c2, i, j, k;
6
7      printf("Enter rows and columns of matrix A: ");
8      scanf("%d %d", &r1, &c1);
9
10     printf("Enter rows and columns of matrix B: ");
11     scanf("%d %d", &r2, &c2);
12
13     printf("Enter elements of matrix A:\n");
14     for (i = 0; i < r1; i++)
15         for (j = 0; j < c1; j++)
16             scanf("%d", &a[i][j]);
17
18     printf("Enter elements of matrix B:\n");
19     for (i = 0; i < r2; i++)
20         for (j = 0; j < c2; j++)
21             scanf("%d", &b[i][j]);
22
23     for (i = 0; i < r1; i++) {
24         for (j = 0; j < c2; j++) {
25             c[i][j] = 0;
26             for (k = 0; k < c1; k++)
27                 c[i][j] += a[i][k] * b[k][j];
28         }
29     }
30
31     printf("Resultant Matrix:\n");
32     for (i = 0; i < r1; i++) {
33         for (j = 0; j < c2; j++)
34             printf("%d ", c[i][j]);
35         printf("\n");
36     }
37
38     return 0;
39 }
```

Problems Output Debug Console Terminal Ports

```
Enter elements of matrix B:
7
5
7
5
8
4
Resultant Matrix:
67 41
133 83
PS E:\OOP LAB>
```

Fig-Input&Output(Matrix Multiplication)

Observation:

- Input: $A (2 \times 2) = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, $B (2 \times 2) = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$
- Output: $\begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$
- Multiplication requires three nested loops, giving $O(r_1 \times c_2 \times c_1)$ time complexity.

Conclusion: Matrix multiplication is possible only when the number of columns of the first matrix equals the number of rows of the second, and each result element is the dot product of a row and a column.

Program No: 12

Program Name: Main Diagonal / Secondary Diagonal Sum

Objective: To write a C program to find the sum of the main diagonal and the secondary (anti) diagonal elements of a square matrix.

Procedure:

1. Start.
2. Read the size n of the square matrix and its elements.
3. Initialize $\text{mainSum} = 0$ and $\text{secSum} = 0$.
4. For each row i , add $\text{arr}[i][i]$ to mainSum (main diagonal).
5. For each row i , add $\text{arr}[i][n-1-i]$ to secSum (secondary diagonal).
6. Print both sums.
7. Stop.

Input&Output:

C Labreport.c X

C Labreport.c > main()

```
1  #include <stdio.h>
2
3  int main() {
4      int arr[10][10], n, i, j, mainSum = 0, secSum = 0;
5
6      printf("Enter size of square matrix: ");
7      scanf("%d", &n);
8
9      printf("Enter elements of matrix:\n");
10     for (i = 0; i < n; i++)
11         for (j = 0; j < n; j++)
12             scanf("%d", &arr[i][j]);
13
14     for (i = 0; i < n; i++) {
15         mainSum += arr[i][i];
16         secSum += arr[i][n - 1 - i];
17     }
18
19     printf("Sum of Main Diagonal = %d\n", mainSum);
20     printf("Sum of Secondary Diagonal = %d\n", secSum);
21
22     return 0;
23 }
```

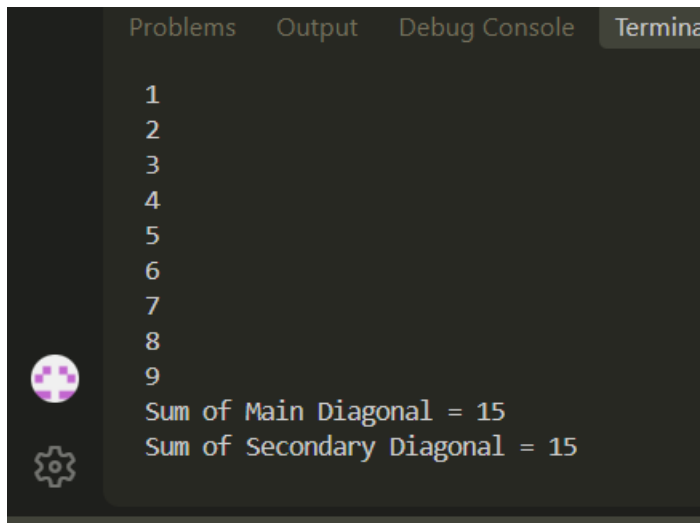
Problems Output Debug Console Terminal Ports

PS E:\OOP LAB> & "E:\OOP LAB\Labreport.exe"

Enter size of square matrix: 3

Enter elements of matrix:

```
1
2
3
4
5
6
7
8
```



```
Problems  Output  Debug Console  Terminal

1
2
3
4
5
6
7
8
9
Sum of Main Diagonal = 15
Sum of Secondary Diagonal = 15
```

Fig-Input&Output(Main Diagonal / Secondary Diagonal Sum)

Observation:

- Input (3x3): $[[1,2,3],[4,5,6],[7,8,9]]$
- Main diagonal: $1+5+9 = 15$
- Secondary diagonal: $3+5+7 = 15$
- Both sums were computed correctly in a single pass over the rows.

Conclusion: The main diagonal elements satisfy $\text{row} = \text{col}$, while the secondary diagonal elements satisfy $\text{col} = n-1-\text{row}$; both sums can be computed in $O(n)$ time.

Program No: 13**Program Name:** Identity Matrix

Objective: To write a C program to check whether a given square matrix is an identity matrix, and/or to generate one.

Procedure:

1. Start.
2. Read the size n of the square matrix and its elements.
3. Set a flag isIdentity = 1.
4. Traverse each element `arr[i][j]`.
5. If `i == j` and `arr[i][j] != 1`, set flag to 0.
6. If `i != j` and `arr[i][j] != 0`, set flag to 0.
7. If flag remains 1, print "Identity Matrix", else print "Not an Identity Matrix".
8. Stop.

Input&Output:

C Labreport.c X

C Labreport.c > main()

```
1  #include <stdio.h>
2
3  int main() {
4      int arr[10][10], n, i, j, isIdentity = 1;
5
6      printf("Enter size of square matrix: ");
7      scanf("%d", &n);
8
9      printf("Enter elements of matrix:\n");
10     for (i = 0; i < n; i++)
11         for (j = 0; j < n; j++)
12             scanf("%d", &arr[i][j]);
13
14     for (i = 0; i < n; i++) {
15         for (j = 0; j < n; j++) {
16             if (i == j && arr[i][j] != 1)
17                 isIdentity = 0;
18             if (i != j && arr[i][j] != 0)
19                 isIdentity = 0;
20         }
21     }
22
23     if (isIdentity)
24         printf("The matrix IS an Identity Matrix.\n");
25     else
26         printf("The matrix is NOT an Identity Matrix.\n");
27
28     return 0;
29 }
```

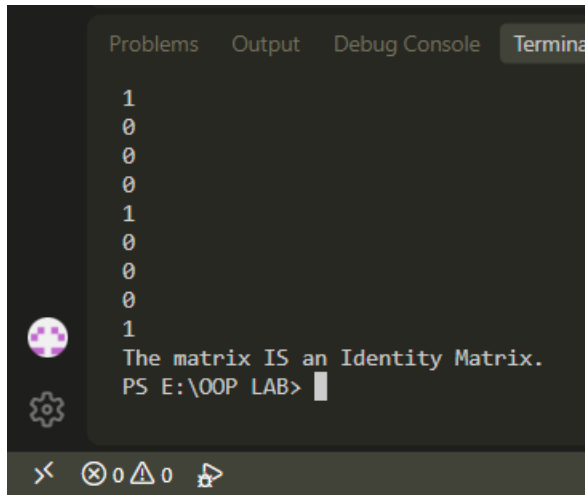
Problems Output Debug Console Terminal Ports

PS E:\OOP LAB> & "E:\OOP LAB\Labreport.exe"

Enter size of square matrix: 3

Enter elements of matrix:

1
0
0
0
1
0
0
0



The screenshot shows a terminal window with a dark background. At the top, there are tabs for 'Problems', 'Output', 'Debug Console', and 'Terminal'. The 'Terminal' tab is active. The input consists of a 3x3 matrix of integers: 1, 0, 0; 0, 1, 0; 0, 0, 1. The output is the text 'The matrix IS an Identity Matrix.' followed by a prompt 'PS E:\OOP LAB>'.

```
1
0
0
0
1
0
0
0
1
The matrix IS an Identity Matrix.
PS E:\OOP LAB>
```

Fig-Input&Output(Identity Matrix)

Observation:

- Input (3x3): $[[1,0,0],[0,1,0],[0,0,1]] \rightarrow$ Output: IS an Identity Matrix
- Input (3x3): $[[1,0,0],[0,2,0],[0,0,1]] \rightarrow$ Output: NOT an Identity Matrix
- The check correctly verifies diagonal elements are 1 and off-diagonal elements are 0.

Conclusion: An identity matrix is a square matrix in which all main diagonal elements are 1 and all other elements are 0; this can be verified with a single traversal comparing row and column indices.

Program No: 14

Program Name: Symmetric Matrix.

Objective: To write a C program to check whether a given square matrix is symmetric, i.e., equal to its own transpose.

Procedure (Algorithm):

1. Start.
2. Read the size n of the square matrix and its elements.
3. Set a flag `isSymmetric = 1`.
4. Traverse each element `arr[i][j]`.
5. If `arr[i][j] != arr[j][i]`, set flag to 0.
6. If flag remains 1, print "Symmetric Matrix", else print "Not Symmetric".
7. Stop.

Input&Output:

```
File Edit Selection View Go Run Terminal Help
C Labreport.c X
C Labreport.c > main()
1  #include <stdio.h>
2
3  int main() {
4      int arr[10][10], n, i, j, isSymmetric = 1;
5
6      printf("Enter size of square matrix: ");
7      scanf("%d", &n);
8
9      printf("Enter elements of matrix:\n");
10     for (i = 0; i < n; i++)
11         for (j = 0; j < n; j++)
12             scanf("%d", &arr[i][j]);
13
14     for (i = 0; i < n; i++) {
15         for (j = 0; j < n; j++) {
16             if (arr[i][j] != arr[j][i])
17                 isSymmetric = 0;
18         }
19     }
20
21     if (isSymmetric)
22         printf("The matrix IS Symmetric.\n");
23     else
24         printf("The matrix is NOT Symmetric.\n");
25
26     return 0;
27 }
```

Problems Output Debug Console Terminal Ports

```
Enter elements of matrix:
1
2
3
2
4
5
3
5
6
The matrix IS Symmetric.
PS E:\OOP LAB>
```

Fig-Input&Output(Symmetric Matrix)

Observation:

- Input (3x3): $[[1,2,3],[2,4,5],[3,5,6]] \rightarrow$ Output: IS Symmetric (matrix equals its transpose)
- Input (3x3): $[[1,2,3],[4,5,6],[7,8,9]] \rightarrow$ Output: NOT Symmetric
- The comparison of $arr[i][j]$ with $arr[j][i]$ correctly identifies symmetry.

Conclusion: A square matrix is symmetric if it is equal to its transpose, i.e., $arr[i][j] = arr[j][i]$ for all i, j ; this property can be verified in $O(n^2)$ time by comparing mirrored elements across the main diagonal.

Program:15

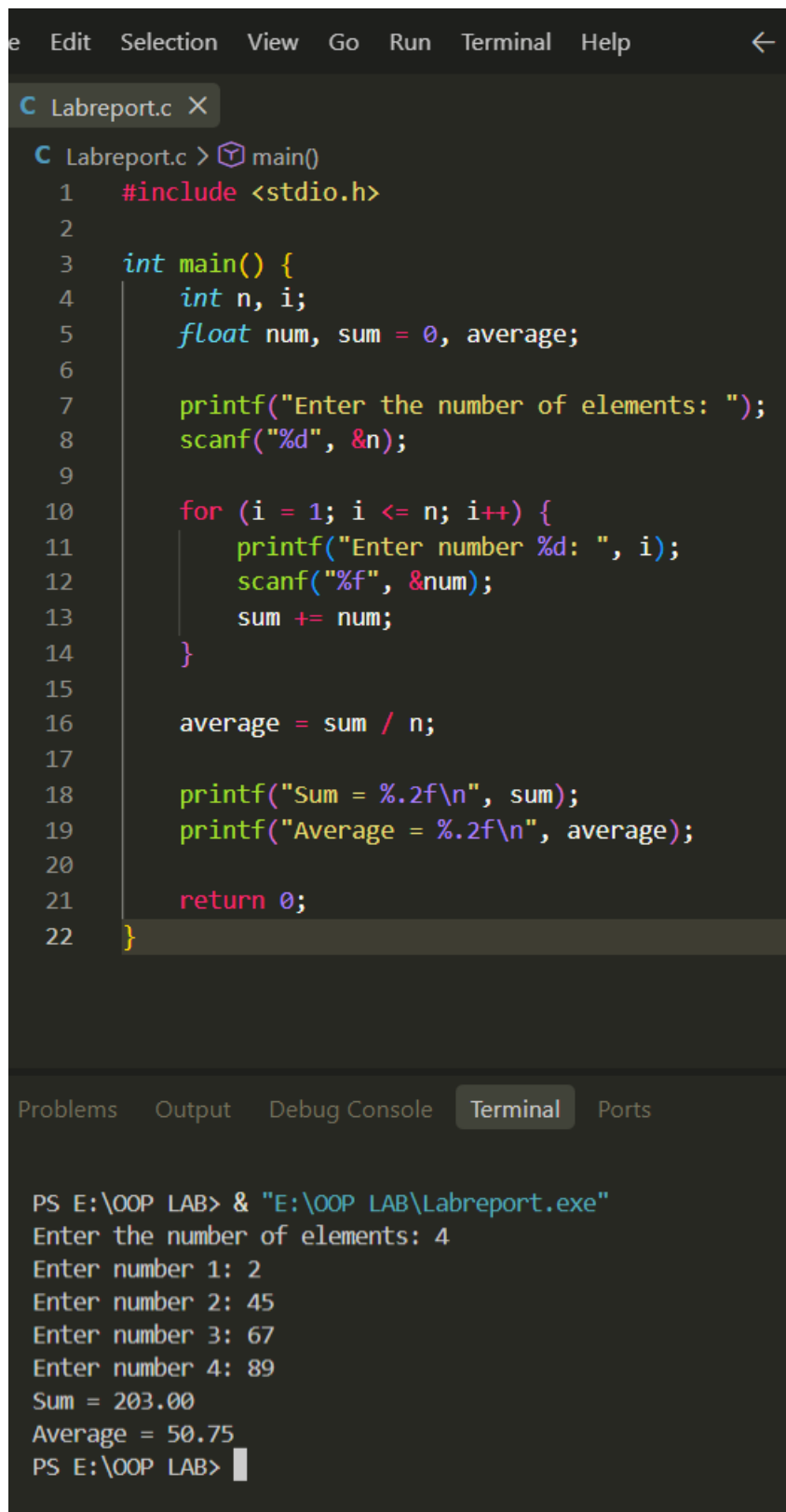
Program Name: Sum and Average of N Numbers

Objective: To write a C program that reads N numbers from the user and calculates their sum and average.

Procedure:

1. Declare variables to store count (n), sum, and average.
2. Prompt the user to enter the value of n.
3. Use a loop to read n numbers and add them to sum.
4. Compute average = sum / n.
5. Display the sum and average.

Input&Output:



The image shows a C++ IDE with a dark theme. The top menu bar includes 'e', 'Edit', 'Selection', 'View', 'Go', 'Run', 'Terminal', and 'Help'. A tab labeled 'Labreport.c' is open. The code editor displays the following C++ code:

```
1  #include <stdio.h>
2
3  int main() {
4      int n, i;
5      float num, sum = 0, average;
6
7      printf("Enter the number of elements: ");
8      scanf("%d", &n);
9
10     for (i = 1; i <= n; i++) {
11         printf("Enter number %d: ", i);
12         scanf("%f", &num);
13         sum += num;
14     }
15
16     average = sum / n;
17
18     printf("Sum = %.2f\n", sum);
19     printf("Average = %.2f\n", average);
20
21     return 0;
22 }
```

Below the code editor, the 'Terminal' tab is active, showing the execution of the program:

```
PS E:\OOP LAB> & "E:\OOP LAB\Labreport.exe"
Enter the number of elements: 4
Enter number 1: 2
Enter number 2: 45
Enter number 3: 67
Enter number 4: 89
Sum = 203.00
Average = 50.75
PS E:\OOP LAB>
```

Fig-Input&Output(Sum and Average of N Numbers)

Observation: The program correctly accepts n numbers and computes their sum and average. Output varies with the values entered by the user. For example, entering 4 numbers (2, 45, 67, 89) gives Sum = 203.00 and Average = 50.75.

Conclusion: The program successfully demonstrates the use of loops and accumulator variables in C to calculate the sum and average of a set of numbers.

Program:16

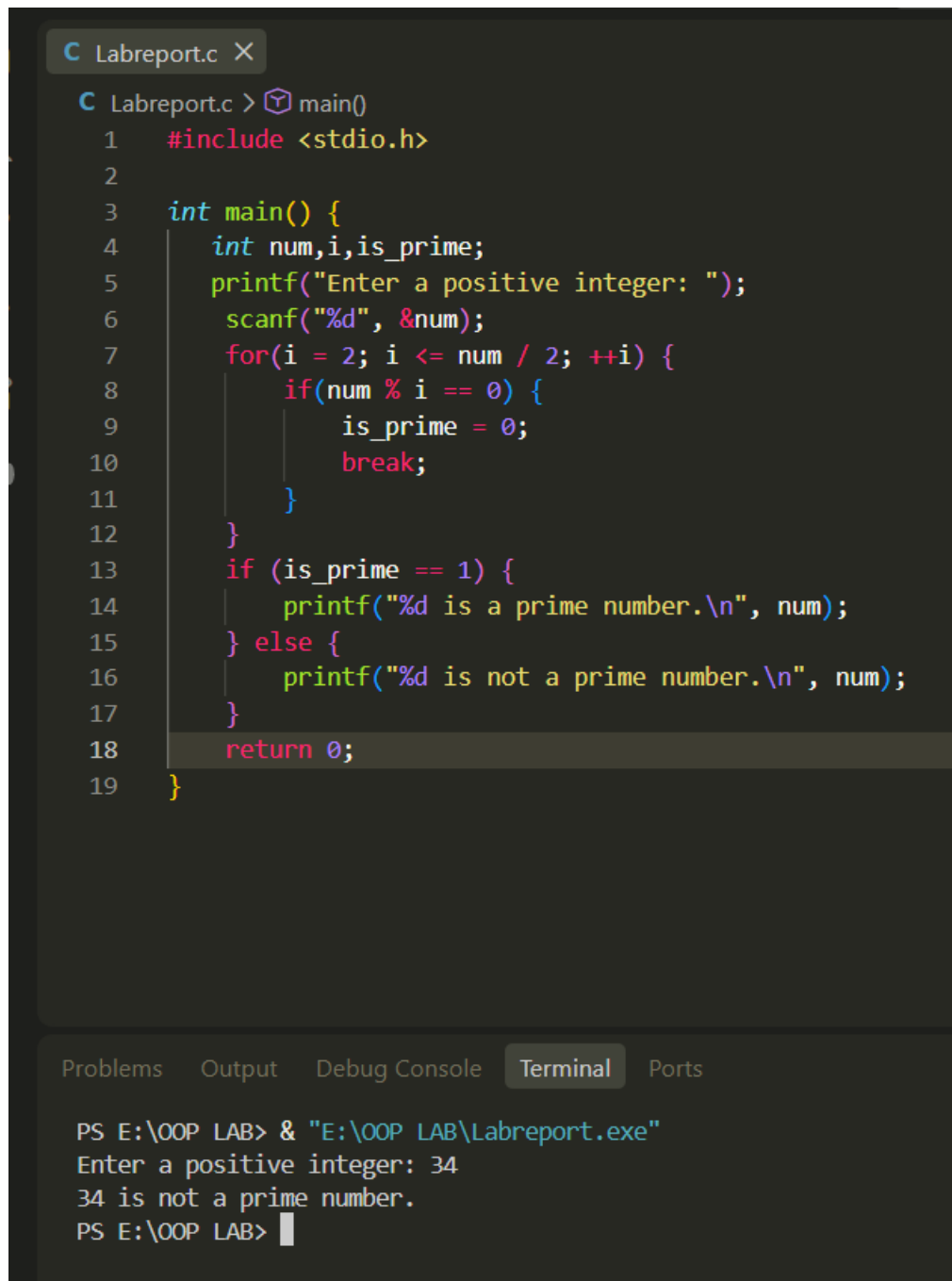
Program Name: Check Whether a Number is Prime

Objective: To write a C program that checks whether a given number is a prime number or not.

Procedure:

1. Read an integer n from the user.
2. Assume the number is prime ($\text{flag} = 1$).
3. Loop from 2 to $n/2$ (or $\sqrt{n}$) and check if n is divisible by any number in that range.
4. If divisible, set $\text{flag} = 0$ and break.
5. Based on the flag, print whether the number is prime or not.

Input&Output:

The image shows a code editor window titled 'Labreport.c' with a C program to check if a number is prime. The code uses a for loop from 2 to num/2 to find divisors. Below the code is a terminal window showing the program's execution with input 34 and output '34 is not a prime number.'

```
C Labreport.c X
C Labreport.c > main()
1  #include <stdio.h>
2
3  int main() {
4      int num,i,is_prime;
5      printf("Enter a positive integer: ");
6      scanf("%d", &num);
7      for(i = 2; i <= num / 2; ++i) {
8          if(num % i == 0) {
9              is_prime = 0;
10             break;
11         }
12     }
13     if (is_prime == 1) {
14         printf("%d is a prime number.\n", num);
15     } else {
16         printf("%d is not a prime number.\n", num);
17     }
18     return 0;
19 }
```

Problems Output Debug Console Terminal Ports

```
PS E:\OOP LAB> & "E:\OOP LAB\Labreport.exe"
Enter a positive integer: 34
34 is not a prime number.
PS E:\OOP LAB> 
```

Fig-Input&Output(Check Whether a Number is Prime)

Observation: For input 7, the output was "7 is a Prime number." For input 12, the output was "12 is NOT a Prime number." The program correctly identifies prime and non-prime numbers.

Conclusion: The program successfully uses conditional statements and looping to determine primality, reinforcing the concept of divisibility checks in C.

Program:17

Program Name: String Reversal Using Pointers

Objective: To write a C program that reverses a given string using pointer manipulation.

Procedure:

1. Declare a character array and accept a string as input.
2. Use a pointer to find the length of the string (or use `strlen()`).
3. Use two pointers — one at the start and one at the end — and swap characters while moving them toward the center.
4. Print the reversed string.
5. End the program.

Input&Output:

```
File Edit Selection View Go Run Terminal Help

C Labreport.c X
C Labreport.c > reverseString(char *)
1  #include <stdio.h>
2  #include <string.h>
3
4  void reverseString(char *str) {
5      char *start = str;
6      char *end = str + strlen(str) - 1;
7      char temp;
8
9      while (start < end) {
10         temp = *start;
11         *start = *end;
12         *end = temp;
13         start++;
14         end--;
15     }
16 }
17
18 int main() {
19     char str[100];
20
21     printf("Enter a string: ");
22     fgets(str, sizeof(str), stdin);
23     str[strcspn(str, "\n")] = '\0';
24
25     reverseString(str);
26
27     printf("Reversed string: %s\n", str);
28
29     return 0;
30 }

Problems Output Debug Console Terminal Ports
PS E:\OOP LAB> cd "E:\OOP LAB\" ; if ($?) { gcc Lab
Enter a string: Muhammad Sumon
Reversed string: nomuS dammahuM
PS E:\OOP LAB> 
```

Fig-Input&Output(String Reversal Using Pointers)

Observation: For the input " Muhammad Sumon ", the output was " nomuS dammahuM ". The pointer-based swapping correctly reversed the string in place.

Conclusion: The program demonstrates effective use of pointers for in-place string manipulation, showing how pointer arithmetic can be used instead of extra array indexing.